

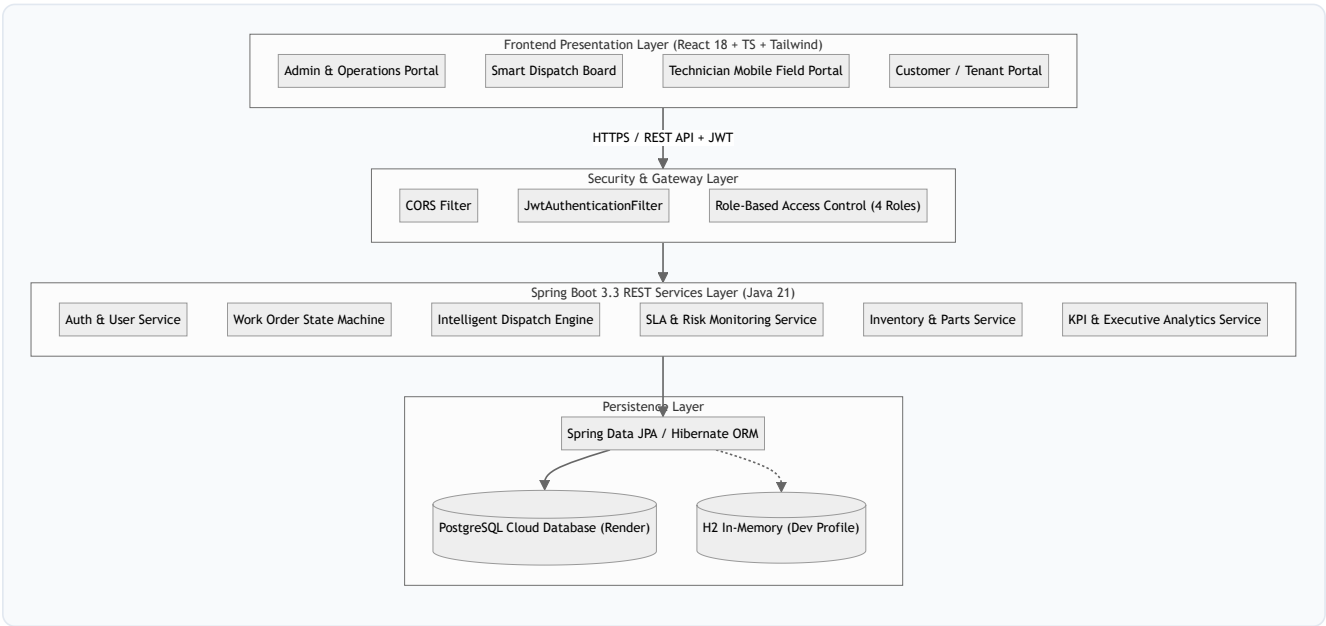
Project Keystone: 6-Month Industrial Training Presentation & Viva-Voce Master Guide

Executive Summary for College Presentation

- **Project Title:** KEYSTONE — Enterprise Field Service Management (FSM) & Operations Platform
- **Domain:** Enterprise Cloud Applications, Facilities Automation, Operations Logistics
- **Role/Context:** 6-Month Industrial Training / Capstone Engineering Project
- **Key Technologies:**
 - **Backend:** Java 21, Spring Boot 3.3.4, Spring Security, Spring Data JPA / Hibernate, JWT (JSON Web Tokens)
 - **Database:** PostgreSQL (Cloud Production on Render) / H2 (Local Development)
 - **Frontend:** React 18, TypeScript, Tailwind CSS, Vite, Lucide Icons
 - **Deployment & Cloud:** Render Cloud (Dockerized JVM Backend & Managed Postgres), Vercel (CI/CD Automated Frontend)

1. System Architecture & Design

High-Level Architecture



2. Core Modules & Technical Innovations

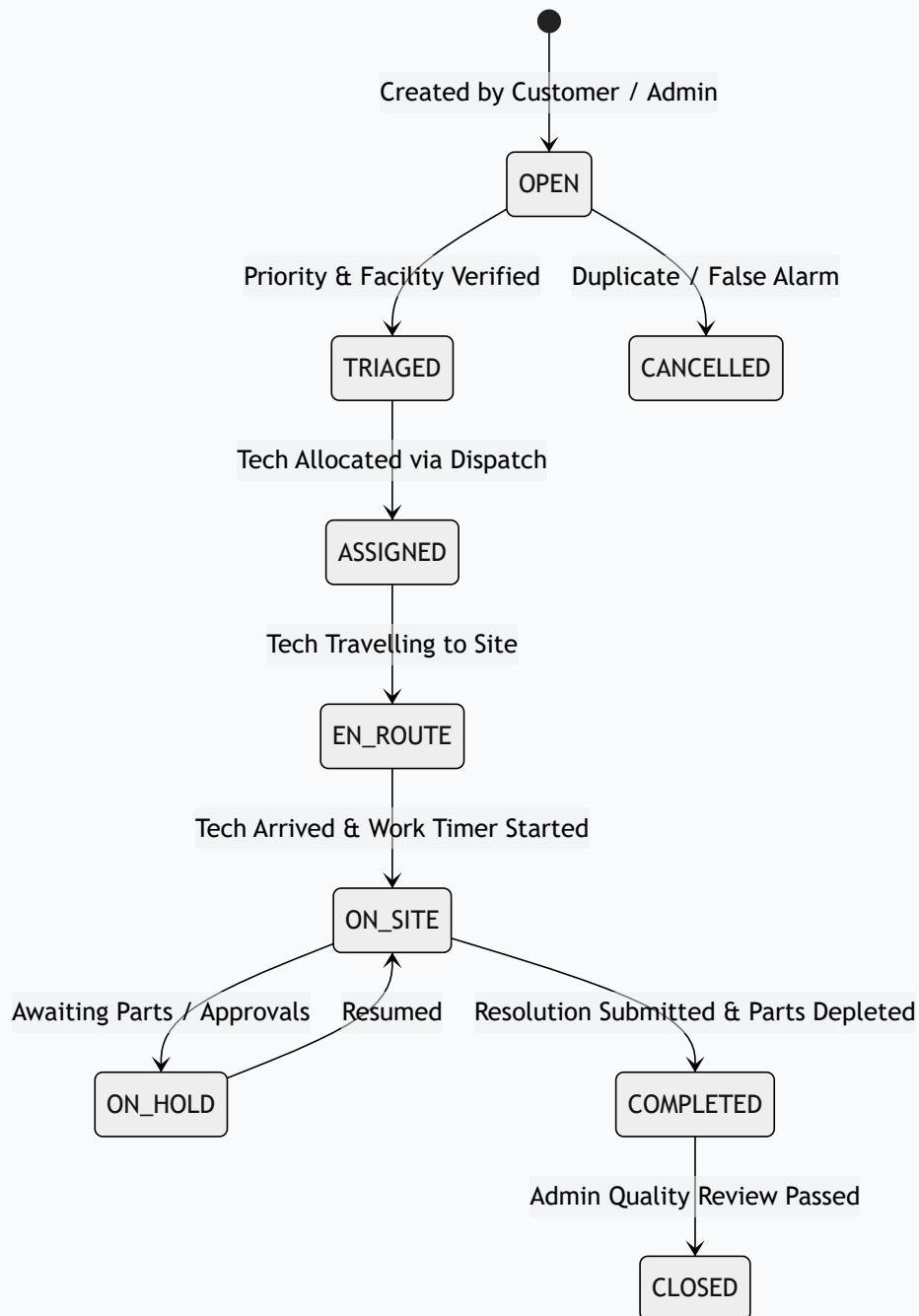
Module 1: Authentication & Multi-Tenant Isolation

- **Stateless Authentication:** Spring Security configured with `SessionCreationPolicy.STATELESS` and custom `JwtAuthenticationFilter`.
- **BCrypt Password Encryption:** Passwords hashed with salted BCrypt before database storage.

- **Admin ID Tenant Isolation:** Commercial customers/tenants register and bind themselves directly to a specific Admin via `adminId`. Customers only view tickets and facilities under their designated administrator.
- **Instant Session Recovery:** Frontend synchronously reconstructs session from encrypted `localStorage` (`keystone_user` + client-side JWT claims), rendering the UI in **0 milliseconds** without blocking cold starts.

Module 2: Work Order Lifecycle State Machine

A rigid 9-stage operational state machine enforces industrial compliance:



Module 3: Intelligent Dispatch & Technician Recommendation Algorithm

Rather than manual assigning, the system computes a **Scoring Metric (0 - 100)**:

$$\text{Score} = \text{Base (40)} + \text{SkillMatch (30-35)} + \text{Availability (+20)} - (\text{ActiveJobs} \times 10) + \text{RatingBonus (+5)}$$

1. **Asset Specialization Match:** Checks technician specialization/certifications against asset category (HVAC, Electrical, Plumbing, Fire Systems).
2. **Current Workload & Availability:** `AVAILABLE` technicians receive $+20$. Active job technicians suffer a -10 penalty per ongoing job. `OFF_DUTY` receives a -50 penalty.
3. **Quality Rating:** High customer rating (≥ 4.8) gives an extra $+5$ bonus.

Module 4: SLA Tracking & Real-Time Risk Engine

Strict time thresholds are enforced based on ticket priority:

Priority	First Response SLA	Resolution SLA	Warning Threshold
CRITICAL	1 Hour	4 Hours	Remaining < 60 min
HIGH	4 Hours	24 Hours	Remaining < 60 min
MEDIUM	8 Hours	48 Hours	Remaining < 60 min
LOW	24 Hours	120 Hours (5 Days)	Remaining < 60 min

- If current time exceeds resolution deadline, ticket is flagged as **BREACHED**.
- In the frontend, dynamic timers update second-by-second with amber pulsing warnings (`WARNING`) or rose alerts (`BREACHED`).

Module 5: Inventory & Spare Parts Depletion

- Track HVAC compressors, refrigerant R-410A, electrical contactors, air filters, valves.
- Real-time stock levels with minimum threshold alert (`quantityOnHand < minThreshold`).
- Automatic inventory decrement upon work order completion.

Module 6: Mobile-First Responsive Architecture

- Single unified codebase serving both desktop operational control room (multi-column tables, analytics cards) and field technician mobile smartphones (touch card views, slide-out hamburger navigation drawers, zero header clutter).

3. Presentation Delivery Script (5 to 7 Minutes)

Slide 1: Introduction & Motivation (1 Minute)

*"Respected Examiners and Professors, Good morning. I am pleased to present **Keystone**, an Enterprise Field Service Management and Facility Maintenance Platform that I designed and built during my 6-month industrial training.*

In commercial facilities—such as high-rise corporate towers, hospitals, and shopping malls—HVAC, electrical, and plumbing failures lead to thousands of dollars in downtime and SLA penalties. Traditional maintenance relies on phone calls, fragmented WhatsApp chats, and paper logs, resulting in delayed dispatching, lost parts inventory, and SLA violations.

Keystone automates the entire lifecycle: from tenant issue logging to algorithmic technician dispatching, real-time SLA countdowns, and automated parts tracking."

Slide 2: Tech Stack & Architecture (1.5 Minutes)

"For the architecture, I adopted an Enterprise 3-Tier Model:

- 1. On the **Backend**, I utilized **Java 21** with **Spring Boot 3.3.4**, using **Spring Security** for stateless **JWT Authentication**, and **Spring Data JPA / Hibernate** for ORM persistence.*
- 2. For the **Database**, I implemented a production-grade relational schema on **PostgreSQL**, deployed on **Render Cloud** with **SSL enforcement**.*
- 3. On the **Frontend**, I built a responsive Single Page Application using **React 18**, **TypeScript**, and **Tailwind CSS**, optimized for both desktop control centers and mobile smartphones.*
- 4. Both tiers are live in production: backend on **Render Cloud** and frontend on **Vercel** with automatic **CI/CD**."*

Slide 3: Key Engineering Features (2 Minutes)

"The platform includes 4 dedicated persona portals:

- First, the **Admin & Operations Portal**: Displays real-time KPIs like **MTTR (Mean Time to Resolution)**, **SLA compliance percentage**, **active work orders**, and **total revenue**.*
- Second, the **Intelligent Dispatch Engine**: An algorithmic matching service that evaluates technician specialization, workload, availability status, and customer rating to calculate a 0 to 100 recommendation match score.*
- Third, the **SLA & Risk Monitor**: Enforces dynamic SLA clocks based on priority (e.g., 4-hour hard deadline for Critical HVAC failures). When a deadline approaches under 60 minutes, the system triggers real-time visual alerts.*
- Fourth, the **Tenant & Technician Portals**: Tenants can submit tickets mapped to their specific facility Admin, while Technicians access a mobile-optimized view with active job timers and parts consumption logs."*

Slide 4: Live Demonstration & Summary (1.5 Minutes)

"Now I will briefly demonstrate the live application: (1) Logging in as Admin, (2) Viewing the live operations board, (3) Opening a Critical work order with its SLA countdown, (4) Utilizing Smart Dispatch to assign the top-ranked technician, and (5) Inspecting the mobile responsiveness.

Thank you, and I am now ready for your questions."

4. Comprehensive Viva-Voce Questions & Model Answers

Category A: Core Java & Spring Boot**Q1: Why did you choose Spring Boot instead of plain Spring MVC?**

*Answer: Spring Boot provides **Opinionated Defaults** and **Auto-Configuration**, eliminating hundreds of lines of boilerplate XML and manual `DispatcherServlet` configuration. It includes an embedded web server (Tomcat), starter dependencies (`spring-boot-starter-web` , `spring-boot-starter-data-jpa` , `spring-boot-starter-security`), and production-ready monitoring.*

Q2: What is the significance of Java 21 in your project?

Answer: Java 21 is a Long-Term Support (LTS) release bringing modern language features such as:

- Pattern Matching for switch expressions (which I used in `SLaService` for calculating priority deadlines).
- Enhanced Virtual Threads (Project Loom) capability for lightweight concurrent I/O.
- Record types and sequenced collections for immutable DTO handling.

Q3: What is the difference between `@RestController` and `@Controller` ?

Answer: `@Controller` is used in traditional MVC where methods return view names resolved by a `ViewResolver` (like JSP or Thymeleaf). `@RestController` is a convenience annotation combining `@Controller` and `@ResponseBody`. Every handler method automatically serializes the returned Java objects directly into JSON via Jackson.

Q4: How does Spring Data JPA simplify database operations?

Answer: Spring Data JPA eliminates boilerplate DAO (Data Access Object) code and raw JDBC SQL statements. By declaring an interface that extends `JpaRepository<WorkOrder, Long>`, Spring dynamically provides CRUD operations, pagination, sorting, and derived query methods like `findByAdminId` or `existsByWorkOrderNumber` at runtime without writing SQL.

Q5: What does `@Transactional` do in your service layer?

Answer: `@Transactional` manages database transactions declaratively. When a method starts, Spring opens a transaction. If the method completes successfully, Spring commits the transaction. If an unchecked exception (`RuntimeException`) occurs (e.g., inventory deduction fails), Spring automatically rolls back all database modifications to preserve data integrity and ACID properties.

Category B: Security & Authentication**Q6: How does JWT Authentication work in your system?**

Answer:

1. The client sends credentials (`email` and `password`) to `/api/auth/Login`.
2. `AuthenticationManager` verifies the credentials using BCrypt.
3. On success, `JwtTokenProvider` generates a signed compact JWT containing user claims (`userId`, `role`, `name`, `sub`) and expiration date, signed with HMAC-SHA256 (`Keys.hmacShaKeyFor`).
4. For subsequent requests, the client attaches the token in the `Authorization: Bearer <token>` HTTP header.
5. `JwtAuthenticationFilter` intercepts the request, validates the signature, extracts the user details, and sets the `SecurityContextHolder` authentication.

Q7: Why did you choose stateless JWT over traditional HTTP Sessions (JSESSIONID)?

Answer:

- **Scalability:** Stateless JWT does not require storing session state in server memory. Multiple backend instances can validate the token independently without sticky sessions or distributed session stores (like Redis).
- **Cross-Origin & Mobile Friendly:** JWT works seamlessly across different domains (CORS), mobile apps, and Single Page Applications (SPAs).
- **CSRF Resilience:** Since JWT is sent via the `Authorization` header rather than automatic browser cookies, it is inherently protected against Cross-Site Request Forgery (CSRF).

Q8: How are passwords secured in the database?

Answer: Passwords are never stored in plaintext. Spring Security's `BCryptPasswordEncoder` uses a one-way hashing algorithm with an automatically generated salt and adjustable work factor (strength: 10). It protects against rainbow table attacks and dictionary attacks.

Q9: What is CORS and how did you configure it?

Answer: CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks web pages from making AJAX requests to a different domain/port. Because our React frontend runs on Vercel (<https://...vercel.app>) and our backend runs on Render (<https://keystone-backend-1usl.onrender.com>), we implemented a `CorsFilter` and `CorsConfigurationSource` in `SecurityConfig` allowing authorized origins, headers (`Authorization` , `Content-Type`), and HTTP methods (`GET` , `POST` , `PUT` , `DELETE` , `OPTIONS`).

Category C: Database, ORM & Data Modeling**Q10: What database schema did you design for Keystone?**

Answer: We implemented a relational schema with key entity tables:

- `users` : Stores user credentials, roles, full names, phone numbers, and optional `admin_id` .
- `facilities` : Commercial sites (address, square footage, contact info).
- `assets` : Equipment inside facilities (HVAC, chiller, electrical panel, QR code).
- `work_orders` : Central operational ticket (WO number, title, priority, status, SLA due timestamps, foreign keys to facility, asset, technician, and tenant).
- `technicians` : Field personnel (skills, certifications, rating, active jobs count, availability status).
- `parts` : Inventory items (part number, unit cost, quantity on hand, min threshold).
- `audit_Logs` : Immutable trail of actions performed on work orders.

Q11: What entity relationships did you use in JPA?

Answer:

- `@ManyToOne` / `@OneToMany`: Between `Facility` and `Asset` (One facility has many assets), and between `Technician` and `WorkOrder` (One technician has many assigned work orders).
- `FetchType.LAZY`: Configured on relational mappings to prevent the N+1 select problem and avoid eagerly loading entire database graphs into memory unnecessarily.

Q12: Why did you use PostgreSQL for production and H2 for development?

Answer:

- **H2** is an in-memory lightweight database ideal for fast local startup and unit testing without requiring local database server installations.
- **PostgreSQL** is an enterprise-grade, ACID-compliant relational database supporting robust JSON operations, advanced indexing, and multi-threaded connection pooling, hosted with SSL encryption on Render Cloud.

Category D: Frontend & User Experience Engineering**Q13: Why did you choose React with TypeScript over plain JavaScript?**

Answer:

- **Type Safety**: TypeScript enforces compile-time typing for all data structures (DTOs, Roles, WorkOrder statuses), preventing common runtime errors like `undefined is not a function`.
- **Refactoring Confidence**: Renaming fields or updating API contracts immediately highlights all affected components.
- **Component-Driven Architecture**: UI elements (Navbar, Cards, Modals, Kanban columns) are modular, testable, and reusable.

Q14: How did you solve the cold-start delay when opening the website?

Answer: On free-tier cloud platforms (like Render), backend containers spin down after 15 minutes of inactivity, taking 30–50 seconds to wake up. Previously, the frontend displayed a blocking `"Connecting to Keystone Platform..."` spinner. We optimized this by:

1. **Synchronous Session Recovery**: Initializing user state synchronously from `localStorage` (`keystone_user` and decoded client-side JWT claims), allowing the dashboard to render **instantly (0ms)**.
2. **Silent Background Sync**: Session validation (`/api/auth/me`) runs non-blockingly in the background.
3. **Keep-Warm Background Ping**: An asynchronous health ping (`/api/health`) fires immediately upon opening the site and periodically every 10 minutes to keep the container awake.

Q15: How did you achieve mobile responsiveness across desktop and mobile devices?

Answer:

- Used **Tailwind CSS breakpoints** (`sm:`, `md:`, `Lg:`).
- **Adaptive Layouts:** On desktop, administrative modules display in an 8-column table with an expanded sidebar. On mobile viewports (`< md`), the table automatically transforms into touch-friendly **Work Order Cards**, and the sidebar becomes a smooth slide-out **Hamburger Drawer**.
- **Clean Mobile Header:** Shifted non-essential header buttons (Logout, Admin ID, Persona Switcher) inside the Hamburger Drawer; preserving ample spacing on small screens (360px–400px).

Category E: Business Logic & Industry Relevance**Q16: How does the technician recommendation algorithm work?**

Answer: Implemented in `DispatchService.recommendTechnicians()`, it calculates a match score:

1. Starts with a base score of 40.
2. Parses the work order asset category and title/description keywords (HVAC, Electrical, Plumbing, Fire Safety). If the technician has matching certifications or specializations, **+30** to **+35** points are added.
3. If the technician is `AVAILABLE`, **+20** points are awarded. If `ON_JOB`, a penalty of $-10 \times \text{activeJobsCount}$ is applied. If `OFF_DUTY`, **-50** points are deducted.
4. If the technician maintains a high customer rating (≥ 4.8), an extra **+5** bonus is granted.
5. Results are clamped between 5 and 100 and sorted in descending order to present the dispatcher with the best candidate.

Q17: What is MTTR and why is it important in your Dashboard?

Answer: **MTTR (Mean Time to Resolution)** is the average elapsed time between ticket creation and completed resolution. It is a critical KPI for commercial facility management. Lower MTTR indicates higher operational efficiency and lower equipment downtime. Keystone calculates and tracks MTTR alongside SLA compliance rate on the Executive Dashboard.

Q18: What are the future enhancements for this project?

Answer:

1. **Real-Time GPS Tracking:** Integrating Leaflet/Mapbox with WebSockets to track field technicians en route in real-time.
2. **Automated Kafka Event Pipeline:** Moving from synchronous dispatching to an event-driven architecture with Apache Kafka for millions of IoT sensor alerts.
3. **Predictive Maintenance with Machine Learning:** Training ML models on asset temperature and vibration telemetry to predict equipment failure before it occurs.

 5. Quick-Reference Cheat Sheet for Viva

Concept	Keystone Implementation
Architectural Pattern	3-Tier Layered Architecture (REST Controller → Service → Repository → Database)
Authentication	Stateless JWT (HMAC-SHA256, 24-hr expiry), BCrypt hashing
Authorization	Spring Security <code>@EnableMethodSecurity</code> , 4 Roles (<code>ROLE_ADMIN</code> , <code>ROLE_DISPATCHER</code> , <code>ROLE_TECHNICIAN</code> , <code>ROLE_CUSTOMER</code>)
SLA Benchmark	Critical: 4h, High: 24h, Medium: 48h, Low: 120h
Database Pool	HikariCP (Spring Boot default high-performance connection pool)
Frontend State	React Context API (<code>AuthContext</code>), optimistic UI updates, zero-latency localStorage cache
Hosting & CI/CD	Backend: Render Cloud Docker JVM. Frontend: Vercel CDN. Git remote: GitHub <code>main</code> .